

Introduction to Unix systems

Lecture 04

Computer Physics

29 October 2026

Processes and resource management

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Faculty of Physics and Applied Computer Science,
AGH University of Krakow

Outline

The Unix process model

Jobs and signals

Process monitoring and priorities

System resources and memory

Section 1

The Unix process model

Programs and processes

- A **program** contains instructions to execute. An executable file stores a program on disk.
- A **process** is an executing instance of a program, together with its current state and resources.
- Each process has an identity, an address space, an execution state and a table of open file descriptors.
- Several processes can execute the same program with different input data and independent private memory.

Example: independent simulation runs

The same `./simulation` executable can serve many runs. Each run has its own process identity, parameters and results.

Process identifiers: PID and PPID

- **PID**: the numeric identifier of a process.
- **PPID**: the PID of its current parent process.
- A process name can occur many times. PIDs distinguish the individual instances.

In an interactive Bash session, `$$` expands to the shell's PID:

```
1 echo "Shell PID: $$"
2 ps -p "$$" -o pid,ppid,comm
```

An illustrative result:

```
Shell PID: 4200
    PID    PPID COMMAND
   4200    4100 bash
```

`-p` selects a PID; `-o` selects the displayed columns. Actual numbers will differ on your system.

Parents, children and the process hierarchy

Processes form a hierarchy through their parent–child relationships. An illustrative fragment of a process tree:

```
bash                  PID 4200  PPID 4100
|-- simulation        PID 4210  PPID 4200
|   `-- worker        PID 4211  PPID 4210
`-- python            PID 4220  PPID 4200
```

- `simulation` and `python` are children of Bash.
- `worker` is a child of `simulation` and a descendant of Bash.
- The shell is itself a process with its own parent. A program can also create children without a shell's involvement.

Reading the hierarchy

A child's PPID matches its current parent's PID. The tree describes relationships, not execution order.

Process states on Linux

A process may exist without currently using a CPU. Common state codes shown by `ps`:

Code	State	Meaning
`R`	Running / runnable	Executing or ready for CPU time.
`S`	Interruptible sleep	Waiting for an event, such as input.
`D`	Uninterruptible sleep	Waiting in the kernel, often for I/O.
`T`	Stopped	Execution suspended by job control.
`Z`	Zombie	Execution finished; termination information awaits collection.

Sleeping and stopped processes

A sleeping process waits for an event and can then become runnable. A stopped process needs to be resumed before execution can continue.

fork: creating a child process

`fork()` creates a child from the calling process. Both continue from the point where the call returns.

- The child receives a new PID. Its initial PPID is the parent's PID.
- Each process has its own address space. Private memory initially has the same contents.
- Linux uses **copy-on-write**: private memory pages can share physical storage until a write requires a separate copy.

Return value	Where / meaning
Child's PID	In the parent: the child was created.
`0`	In the child: execution continues here too.
`-1`	In the parent: creation failed; no child exists.

Scheduling after fork

Either process may run first. The parent and child can make progress independently.

exec: replacing the running program

The `exec` **family** loads a new program into the calling process. `execve()` is the Linux system call.

Property	Before exec	After successful exec
PID	`4210`	`4210`
PPID	`4200`	`4200`
Program	Child running shell code	`wc`
Program memory	Shell's code and data	New program's code and data

- The caller supplies the new program's arguments and environment.
- A successful call continues in the new program. It never returns to the old program's next instruction.
- An error return lets the calling program handle a failed replacement.

Separate roles

`fork()` creates a process. `exec` changes the program executed by an existing process.

File descriptors belong to a process

A **file descriptor** is a small non-negative integer indexing an entry in a process's table of open descriptors.

Descriptor	Conventional role	Possible connection
`0`	Standard input	Terminal, input file or pipe.
`1`	Standard output	Terminal, output file or pipe.
`2`	Standard error	Terminal or diagnostic log.
`3`, `4`, …	Additional descriptors	Other open files, pipes or sockets.

- A descriptor identifies an open connection, not a pathname.
- The same number in two processes can refer to different objects.

Connection to Lecture 03

Redirection changes descriptor connections. Writing to descriptor `1` can send output to a file while the program continues to use stdout.

File descriptors across fork and exec

After `fork()`, the child inherits copies of the parent's open descriptors. An illustrative example:

Descriptor	Parent's table	Child's table
`0`	Terminal input	Same terminal input
`1`	Terminal output	Same terminal output
`3`	Open record A: `input.dat`	Same open record A

- Separate tables refer to the same kernel **open file descriptions** (such as record A).
- For a regular file, reads in either process advance the same current file position (offset).
- Closing the child's descriptor leaves the parent's entry open.

What survives exec?

Open descriptors remain available to the new program, except those marked `FD_CLOEXEC` (close-on-exec).

How the shell launches an external command

Reuse `data/motion.txt` from the earlier lectures:

```
1 wc -l < data/motion.txt > count.txt
```

A typical model for an external command in interactive Bash:

1. **Create a child:** the shell uses `fork()` or an equivalent process-creation mechanism.
2. **Prepare its descriptors:** connect `0` to `data/motion.txt` and `1` to `count.txt`. Descriptor `2` keeps its inherited connection.
3. **Replace the child's program:** use `exec` to run `wc` with argument `-l`.
4. **Wait in the parent:** the shell waits for completion, collects the result and displays the next prompt.

The parent shell keeps its identity

The child runs `wc`; the interactive shell remains available for later commands. Builtins such as `cd` normally execute in the shell itself.

Process termination, zombies and orphans

- When a process terminates, the kernel releases its program memory and closes its descriptors.
- The parent can collect termination information with `wait()` or `waitpid()`. For a normal exit, this includes the child's exit status.
- Under normal handling, an exited child remains a **zombie** until its parent collects that information. This collection is called **reaping**.
- If the parent exits first, its child becomes an **orphan**. Linux reparents it to a designated reaper: a suitable subreaper or the namespace's init process (PID 1).

Zombie and orphan describe different situations

A zombie has finished executing and retains a small kernel record. An orphan has lost its original parent and may still be running.

Example: the PID survives exec

`bash -c` runs the quoted commands in a new Bash process. Inside it, the `exec` builtin replaces that Bash with `ps`:

```
bash -c 'echo "Before exec: PID=$$"
exec ps -p "$$" -o pid,ppid,comm'
```

An illustrative result:

```
Before exec: PID=4210
    PID    PPID COMMAND
   4210    4200 ps
```

- The program changes from Bash to `ps`; PID `4210` stays the same. Actual PIDs will differ.
- The original interactive shell remains the parent.

Predict: if you append `> identity.txt` to the complete command, where will the message and the table appear?

Section 2

Jobs and signals

Foreground and background jobs

In interactive Bash, a **job** is a command or pipeline managed by the shell. Its processes form a process group.

Job	Relation to the shell and terminal
Foreground	The shell waits until the job finishes or stops. The job can read terminal input.
Background	The shell accepts further commands while the job runs. Terminal input normally belongs to the foreground group.

- The pipeline `sleep 300 | cat` contains two processes but forms one job.
- These examples assume an interactive Bash session with job control enabled.

Connection to process states

Foreground and background describe a job's relation to the terminal. A background process may be runnable, sleeping or stopped.

Starting and listing background jobs

A trailing `&` starts a job in the background:

```
sleep 300 &
jobs -l
```

An illustrative result in a shell with no other jobs:

```
[1] 4210
[1]+  4210 Running                 sleep 300 &
```

- `[1]` is the shell's job number. Refer to this job as `%1`; `+` marks the current job.
- `4210` is the PID of `sleep`. `jobs -l` includes PIDs as well as job status.
- `$!` gives the PID of the last command in the most recently started background pipeline: here, `sleep`.

Job numbers are local to the shell

`jobs` lists this shell's jobs. A second terminal has its own job table, so its `%1` can identify a different job.

Ctrl+C and Ctrl+Z: interrupt or suspend

With the usual terminal settings, these keys signal the whole **foreground process group**:

Key	Signal	Default action
`Ctrl+C`	`SIGINT`	Terminate execution.
`Ctrl+Z`	`SIGTSTP`	Suspend execution for later resumption.

In a fresh shell with no jobs, run `sleep 300`, then press `Ctrl+Z`:

```
$ sleep 300
^Z
[1]+  Stopped                 sleep 300
```

`^Z` represents the key press, not a command to type.

Suspending preserves the process

A stopped process retains its PID and memory and can resume later.

Resuming jobs with bg and fg

Continue the stopped `sleep` job from the previous example. Enter commands one at a time:

```
$ bg %1
[1]+ sleep 300 &
$ jobs
[1]+  Running                 sleep 300 &
$ fg %1
sleep 300
^C
$
```

- `bg %1` resumes the stopped job in the background. The shell accepts the next command immediately.
- `fg %1` brings the job to the foreground, resuming it if necessary. Here, `Ctrl+C` then ends `sleep`.

Resuming continues the existing processes

`bg` and `fg` preserve PIDs and the program's progress. They do not start a new run of the command.

Signals: notifications with defined actions

A **signal** notifies a process of an event. The kernel or another process can generate it. These signals have the following default actions:

Signal	Default action	Typical purpose
`SIGINT`	Terminate	Interactive interruption.
`SIGTERM`	Terminate	Request termination.
`SIGKILL`	Terminate	Force termination.
`SIGTSTP`	Stop	Suspend from the terminal.
`SIGSTOP`	Stop	Force suspension.
`SIGCONT`	Continue	Resume a stopped process.

A program can handle many signals with its own function (a **signal handler**) or choose to ignore them.

Two signals the application cannot override

`SIGKILL` and `SIGSTOP` cannot be caught, blocked or ignored. `SIGCONT` resumes a stopped process.

Sending signals with kill

`kill` sends a signal. With no signal option, it sends `SIGTERM`. In Bash, `kill` is normally a shell builtin.

Example	Target
`kill -TERM 4210`	The single process with PID `4210`.
`kill -TERM %1`	All processes in this shell job's process group.

A separate example using the PID of a job you have just started:

```
sleep 300 &
sleeper_pid=$!
kill -TERM "$sleeper_pid"
```

`kill -s TERM` is another spelling of `kill -TERM`. Use `kill -l` to list signal names.

A PID target and a job target can have different scope

For a pipeline, signalling one PID may leave other members running. You can normally signal processes owned by your account.

Terminating a job: TERM before KILL

For a running background job shown as `%1`, first request termination:

```
kill -TERM %1
jobs -l
```

- `SIGTERM` allows an application to handle the request, for example by flushing output and saving its state before exiting.
- Allow time for termination and check the job and its output. Successful signal delivery does not prove that the job has finished.

Only if the same job remains and normal termination has failed:

```
kill -KILL %1
```

Forced termination skips application cleanup

With `SIGKILL` (the `-9` option), the program cannot save buffered results or run a cleanup handler. The kernel still releases its resources.

Background jobs and terminal input/output

A background job normally inherits stdout and stderr, so its messages can appear among your commands and prompts.
Assume `./simulation` reads its parameters from stdin:

```
./simulation < input.dat > run.log 2>&1 &
```

- `< input.dat` supplies input without reading the terminal.
- `> run.log 2>&1` sends stdout and stderr to the log. The final `&` starts the job in the background.
- With job control enabled, a background process that tries to read the terminal normally receives `SIGTTIN` and stops. Use `fg` when a job needs interactive input.

Background execution and session lifetime

`&` does not detach the job from the login session or protect it from hangup. Closing the terminal or losing the connection can end background work.

Practice: control one job through its lifetime

Work in one interactive Bash session with no existing jobs. Enter each command separately and complete the exercise within five minutes.

```
sleep 300 &
jobs -l
```

1. Bring the job to the foreground with `fg %1`, then suspend it with `Ctrl+Z`.
2. Run `jobs -l`, resume it with `bg %1`, then inspect `jobs -l` again.
3. Bring it to the foreground once more and press `Ctrl+C`. Inspect the job table after it ends.

Check your reasoning

Which actions change the PID? Which change the execution state? While the job is in the background, would `Ctrl+C` at the shell prompt interrupt it?

Section 3

Process monitoring and priorities

Inspecting processes with ps and pstree

`ps` reports a snapshot of selected processes:

```
ps -u "$USER" -o pid,ppid,stat,comm
```

`-u` selects a user; `-o` selects columns. Here: process ID, parent ID, state and command name.

`pstree` displays parent–child relationships:

```
pstree -p "$$"
pstree -sp "$$"
```

- `-p` includes PIDs. The first tree starts at your shell.
- `-s` also shows its ancestors, tracing the path to the shell.

A wider view than the shell's job table

`jobs` tracks the current shell's jobs. `ps` can select processes from other terminals too.

Choosing columns and sorting with ps

List your processes with the largest CPU percentages first:

```
ps -u "$USER" -o pid,stat,pcpu,pmem,rss,ni,comm --sort=-pcpu
```

An illustrative result:

```
  PID STAT %CPU %MEM    RSS  NI COMMAND
 4210 R    97.1  1.6 262144   0 simulation
 4220 S     0.3  0.1  24576  10 python
```

- `%MEM` expresses resident memory (`RSS`) as a percentage of the machine's physical memory.
- `NI` is the nice value. `STAT` starts with the process state, possibly followed by extra flags.
- The minus in `--sort=-pcpu` requests descending order. Use `--sort=-rss` to sort by resident memory.

CPU percentage and current state describe different things

`STAT` is the state observed now. `ps` `%CPU` averages CPU use over the process's lifetime.

Live monitoring with top and htop

`top` refreshes process information repeatedly. To restrict its process list to your user:

```
top -u "$USER"
```

Useful keys **inside top**:

Key	Action
`P`	Sort by CPU usage.
`M`	Sort by memory usage.
`1`	Toggle the summary for individual logical CPUs.
`h`	Show help.
`q`	Quit the viewer.

An alternative interactive viewer

`htop`, when installed, provides another interactive process viewer. Run `htop` and use its on-screen key labels. Quitting either viewer leaves the monitored processes running.

Interpreting CPU and memory figures

- **Measurement interval:** `ps %CPU` is a lifetime average. `top %CPU` normally covers the last refresh interval.
- **CPU scaling:** in `top`'s usual per-CPU mode, `100%` means one logical CPU fully occupied. A process using several threads can exceed `100%`.

Field	Meaning
`RSS`	Resident physical memory, including shared pages.
`VSZ`	Virtual address-space size, not the amount of RAM occupied.

In `ps`, `RSS` and `VSZ` use KiB (`1024` bytes). `top` can scale its memory units.

A large number needs context

High CPU use can be expected during computation. Shared pages mean that adding process RSS values can count the same physical memory more than once.

CPU scheduling and the nice value

The scheduler chooses which runnable tasks receive CPU time. For normal scheduling, the **nice value**, shown as `NI`, influences their relative preference.

Nice value	Scheduling preference
`-20`	Most favourable within the nice range.
`0`	Usual starting value.
`19`	Least favourable within the nice range.

- A higher number gives competing work more preference.
- Scheduling groups and system resource policies also affect how CPU time is shared.

Niceness is a scheduling preference, not a CPU limit

A process with a high nice value can still fully occupy a CPU when no other runnable work competes for it.

Starting with nice and adjusting with renice

Assume a single-threaded `./simulation` is available and the shell has nice value `0`. Start a run with a higher nice value:

```
nice -n 10 ./simulation < input.dat > run.log 2>&1 &
run_pid=$!
```

`nice -n 10` **adds 10** to the inherited value, giving `NI=10` here.

While that process is still running, set its nice value to `15`:

```
renice 15 -p "$run_pid"
ps -p "$run_pid" -o pid,ni,comm
```

With Linux `renice`, the first number in this form is the **new absolute value**, not an increment.

Permissions and scope

An ordinary user can normally only increase the nice value of their own processes. Lowering it usually needs extra privileges. Linux niceness is per thread: other existing threads and job members are unaffected by renicing the main PID.

Practice: observe and adjust one process

Check that `nice` prints `0`. Enter commands separately, within five minutes. Before `renice`, confirm that `ps` shows `sleep` with `NI=10`.

```
nice -n 10 sleep 300 &
monitor_pid=$!
ps -p "$monitor_pid" -o pid,stat,ni,comm
renice 15 -p "$monitor_pid"
ps -p "$monitor_pid" -o pid,stat,ni,comm
kill -TERM "$monitor_pid"
```

- Compare the two snapshots: the PID stays the same, while `NI` changes from `10` to `15`.
- `sleep` normally waits in state `S`. This exercise demonstrates niceness, not CPU contention.

Check your reasoning

Would `NI=15` prevent a CPU-intensive program from using a whole CPU? Would an ordinary user normally be able to change this process back to `NI=0`?

Section 4

System resources and memory

System resources and your allocation

A running program depends on several finite resources:

Resource	What a program uses
CPU	Processing time on logical CPUs.
RAM	Resident memory for instructions and data.
Storage and I/O	Space for files and bandwidth for data transfers.
File descriptors	Open files, pipes and sockets.

`nproc` reports processing units available to the current process:

```
1 nproc
```

Visible capacity and assigned resources can differ

On a shared node or in a container, host totals may exceed your allocation. Check the job's assigned CPUs and memory as well as the machine's capacity.

Virtual memory and physical RAM

Programs use **virtual addresses**. The kernel manages mappings between virtual pages and physical RAM.

- A process has its own virtual address space. Separate spaces help protect processes from each other.
- Memory is managed in blocks called **pages**. Mappings can also let processes share selected pages.
- With **demand paging**, pages become resident as needed. Reserving a virtual range does not mean that all of it occupies RAM.
- A **page fault** can be normal: the kernel may need to provide a page or load its contents before execution continues.

Connection to the previous section

`VSZ` describes virtual size; `RSS` describes resident memory. These quantities can differ substantially.

Page cache, swap and memory pressure

- Linux uses RAM for a **page cache** to speed up file I/O. It can reclaim clean cached pages when other work needs memory.
- **Swap** provides backing space for some memory pages, often on a disk or SSD. Those pages can leave resident RAM.
- Accessing a swapped-out page requires making it resident again. Heavy, repeated page movement can greatly slow a computation.
- Under severe memory pressure, allocations can fail or the **out-of-memory (OOM) killer** may terminate processes.

A job can exhaust its own memory allowance

A container or job memory limit can be reached while the host still has available RAM. Swap does not provide the performance of additional physical memory.

Reading memory usage with free

Display RAM and swap totals with automatically scaled units:

```
free -h
```

Example output; values depend on the machine:

```
               total        used        free      shared  buff/cache   available
Mem:            27Gi       9.3Gi       9.5Gi       333Mi       7.8Gi        17Gi
Swap:          8.0Gi        82Mi       7.9Gi
```

- `free`: RAM currently unused.
- `available`: estimated RAM for new work without swapping, including reclaimable memory.
- `buff/cache`: buffers and caches. A substantial cache is a normal use of RAM.

Used swap and active swapping are different

Nonzero swap usage does not by itself show current memory pressure. Check whether pages are moving in or out over time.

Observing activity with vmstat

Request five reports, with one second between updates:

```
vmstat 1 5
```

Field	Meaning
`r`	Runnable tasks: executing or waiting for CPU time.
`b`	Tasks blocked waiting for I/O.
`si / so`	Swap-in and swap-out rates.
`us / sy / id`	CPU time: user code, kernel code and idle.
`wa`	CPU time accounted as waiting for I/O.

The first row's rates cover time since boot; later rates cover one-second intervals. Process counts and memory values are snapshots. Use `vmstat -y 1 5` to omit the initial report.

Interpret several observations together

Repeated swap transfers with low available RAM can explain a slowing application.

Resource limits with ulimit

Bash's `ulimit` inspects or changes resource limits. Show all current soft limits:

```
ulimit -a
```

Command	Reported limit
`ulimit -Sn`	Soft limit for open file descriptors.
`ulimit -Hn`	Hard limit for open file descriptors.
`ulimit -Sv`	Virtual address-space size, in KiB.
`ulimit -St`	Accumulated CPU time, in seconds.

The **soft limit** is enforced. An ordinary user can raise it up to the **hard limit**, but cannot normally raise the hard limit.

Limits follow the process hierarchy

Commands started by the shell inherit its limits. `-v` limits virtual size, not resident RAM; `-t` limits CPU time, not elapsed wall-clock time.

Practice: inspect capacity, activity and limits

Run these commands in the same session and record the results:

```
nproc
free -h
vmstat -y 1 5
ulimit -Sn
ulimit -Hn
```

1. Why can `available` be much larger than `free`?
2. If swap is used but `si` and `so` stay near zero, does that show heavy ongoing swapping?
3. If the soft descriptor limit is `1024` and the hard limit is `4096`, could an ordinary user raise the soft limit to `2048`?

A termination message needs supporting evidence

`Killed` alone does not establish an OOM event. Check the program and job logs, including any resource-limit report.